

Seminar Report

RAG-Driven Data Cleaning with PyDI

Marmee Pandya

July 1, 2026

A missing model number like V375-040R or a storage speed of 3480 MB/s cannot be guessed. These values are precise, product-specific facts that carry no signal in a product title or description; they exist only as structured fields in another retailer’s listing of the same item. Large language models face a hard ceiling on exactly this class of attributes, and retrieval-augmented generation offers a principled solution. Whether it holds up inside a focused, single-category product knowledge base, where near-identical hardware variants differ by a single character in a model number, has received limited empirical attention in the literature.

Following RetClean ([Naeem et al. 2024](#)) Scenario 3, this report builds and evaluates a fully local RAG pipeline within PyDI, testing seven configurations across 96 tasks spanning attributes from common vocabulary fields to exact manufacturer SKUs and physical dimensions. The central finding is that CrossEncoder re-ranking delivers the largest single accuracy gain, a 32-point jump from 36.5% to 68.8%, dwarfing the gain from a stronger embedding model. Attributes with no signal for an LLM, such as `height_mm`, rise from 0% to 77% with retrieval. A less expected result: GPT-5.4-mini appears to trail every Llama configuration under standard evaluation, but this is a prompt calibration failure, not a capability gap. Forced to commit, it reaches 77.1%, the highest result overall, at the cost of sending product data to an external API and breaking the privacy constraint. For a fully on-premise deployment, BGE-large with CrossEncoder re-ranking and Llama 3.1 8B at 72.9% is the recommended configuration.

1 Introduction

Real-world product catalogues are rarely complete. In e-commerce datasets it is common to find records where attributes such as bus type, model number, read speed, or physical dimensions are simply absent. Sellers do not always fill every field, scraping

pipelines miss structured specifications, and products from different sources follow different schema conventions. These gaps create practical problems for downstream tasks including product matching, search ranking, and data integration. The most direct remedy is to find the missing value in a knowledge base that describes the same product from another source, a retrieval problem adjacent to entity matching (Peeters and Bizer 2023; Papadakis et al. 2021) but applied here to attribute-level imputation under enterprise privacy constraints, a setting those works do not address.

To illustrate, consider a product record for an MSI GeForce GTX 1660 Ti GPU with a missing `model_number` field. The correct value, `V375-040R`, appears nowhere in the product title or description, it is a manufacturer-assigned identifier that exists only as a structured field in other retailer listings of the same product. No LLM can reliably produce this value from parametric memory alone; it must be looked up.

The problem is representative of a broader class of knowledge missingness in product data: attributes that are product-specific, precisely defined, and present in structured KB fields but absent from free-text descriptions. Classical data cleaning approaches handle statistical missingness well but are not designed for this regime. LLM-based approaches handle common knowledge well but fail precisely where knowledge is product-specific. Retrieval-based approaches address both limitations in principle. RetClean (Naeem et al. 2024) demonstrated this on a manually constructed heterogeneous data lake, but has not been evaluated on a focused single-category product knowledge base where near-identical product variants with nearly identical descriptions make retrieval precision especially demanding. It is also not yet clear whether the privacy cost of cloud-hosted retrieval components is justified by the accuracy gain, or whether a fully local pipeline can close the gap.

This report addresses three questions. First, how much does retrieval improve over LLM-only prediction, and which attribute types benefit most? Second, does a two-stage retrieve-then-rerank pipeline as proposed by RetClean (Naeem et al. 2024) translate into meaningful accuracy gains for a focused product knowledge base? Third, does a stronger cloud-hosted LLM close the remaining gap, and what are the privacy implications of using one within a Scenario 3 deployment?

The pipeline is implemented as a **RAGCleaner** component extending PyDI’s **BaseCleaner** interface, making retrieval-augmented imputation composable with entity matching, schema mapping, and other stages in a PyDI data integration workflow. This architectural choice also enables a fourth experimental question: whether PyDI’s **DataFusionEngine** which consolidates multiple retrieved KB candidates into a single clean record using attribute-specific aggregation strategies can improve accuracy or reduce abstention compared to presenting raw KB rows to the LLM. The fused-record variant isolates the contribution of structured candidate consolidation from the retrieval and prediction steps, and its results are reported in Section 4.4.

The remainder of this report is organised as follows: Section 3 describes the experimental design; Section 4 presents results; Section 5 analyses failures; and Section 6 concludes.

2 Related Work

This section surveys the methods most relevant to RAG-based missing value imputation: classical statistical approaches, LLM-based cleaning, retrieval-augmented generation, and the RetClean system that directly inspires this work.

Classical imputation: Statistical methods such as mean or mode substitution and multiple imputation by chained equations (Rahm and Do 2000; van Buuren 2018) perform well when values follow a distribution. Product identifiers model numbers, hardware specifications, physical dimensions are exact strings that cannot be estimated from neighbouring records, a distinction central to entity resolution (Papadakis et al. 2021).

LLMs for data cleaning: Large language models introduced a new possibility, using parametric knowledge accumulated during pre-training to fill missing values without any external lookup. Narayan et al. (2022) showed that GPT-3 can detect and correct tabular data errors through few-shot prompting, and Zhang et al. (2023) extended this to a broader range of cleaning tasks. Both results are encouraging for attributes that appear frequently in pre-training corpora such as common interface types, brand names, broad product categories. The evidence suggests, however, that this approach reaches a hard ceiling for attributes requiring precise, product-specific knowledge. A model like Llama 3.1 8B Instruct has been exposed to large volumes of product text during pre-training and can sometimes infer missing values directly from a product description, but for attributes requiring exact knowledge the LLM simply does not have access to the right information and will either hallucinate or abstain. This limitation is not a matter of model size; it reflects the practical inability to reliably recall product-specific facts that were never reliably present in training data to begin with, are outdated for recently released products, or are inconsistent across the web sources the model was trained on.

Retrieval-Augmented Generation: RAG (Lewis et al. 2020) grounds LLM outputs in evidence retrieved at inference time, replacing parametric recall with KB lookup. Izacard and Grave (2021) extended this with Fusion-in-Decoder, which attends over multiple retrieved passages jointly rather than processing them independently, demonstrating that retrieval quality directly bounds generation quality. In their setting, as in ours, the retriever’s ability to surface the correct passage is the binding constraint rather than the generator’s capacity. Recent surveys (Gao et al. 2023; Zhao et al. 2024) consistently identify retrieval precision as the dominant bottleneck across RAG applications, and specifically highlight the gap between dense and sparse retrieval on semantic paraphrase tasks as a key motivator for embedding model choice. Cross-domain retrieval benchmarks (Thakur et al. 2021) further show that the advantage of dense over sparse retrieval varies by domain, motivating the embedding comparison in this work. Applied to structured data settings, both LLM-augmented table models (Li et al. 2023) and data engineering surveys (Bodensohn et al. 2024) observe that factual precision requirements for exact structured fields exceed what parametric knowledge alone provides, making

retrieval indispensable. This directly motivates the three-way embedding comparison MiniLM, BGE-large, and OpenAI TE-3-large and the use of CrossEncoder re-ranking to improve candidate precision beyond cosine similarity alone.

Dense retrieval and re-ranking: Bi-encoder architectures (Reimers and Gurevych 2019; Karpukhin et al. 2020) independently encode queries and documents into a shared vector space, enabling efficient nearest-neighbour retrieval that outperforms sparse BM25 (Robertson and Zaragoza 2009) under semantic paraphrasing. Cross-encoder models jointly encode query-document pairs with full cross-attention, yielding more precise relevance scores at higher cost (Nogueira and Cho 2019). The standard two-stage pipeline wide bi-encoder recall followed by cross-encoder reranking is the approach adopted here and directly motivates the reranker ablation in Section 4.

RetClean: RetClean (Naeem et al. 2024) is the most closely related prior work and the primary design reference for this report. Its retrieval pipeline pairs an initial candidate stage using FAISS (Douze et al. 2024) or Elasticsearch with ColBERT (Khattab and Zaharia 2020) or cross-encoder reranking, feeding a cloud-hosted LLM for pairwise tuple matching and value extraction. RetClean defines three deployment scenarios: fully cloud-hosted inference (Scenario 1), cloud LLM with local retrieval (Scenario 2), and fully local model and retrieval for privacy-sensitive settings (Scenario 3). This report implements Scenario 3 with Llama 3.1 8B Instruct (Dubey et al. 2024) via Ollama and few-shot prompting in place of RetClean’s optional fine-tuning step, isolating retrieval quality from fine-tuning effects.

The system is implemented as a **RAGCleaner** component within PyDI,¹ the Python Data Integration framework developed at the University of Mannheim, making it a composable drop-in for any PyDI pipeline. The full experimental code, evaluation scripts, and results are publicly available.²

3 Experimental Design

3.1 Problem Setting

Let D be a product dataset with missing attribute values and KB a knowledge base of product records from other sources. Products across D and KB are linked by a `cluster_id` field identifying records that describe the same real-world product. The goal is to fill missing values in D by retrieving semantically similar products from KB and using an LLM to extract the correct attribute value from the retrieved context.

Ground truth for a given query row and target attribute is defined as the first non-null value found in any KB row sharing the same `cluster_id`. Empirical verification confirms that all 96 evaluation tasks have their ground truth present in at least one KB row (100% coverage), ensuring that no task penalises the system for KB incompleteness.

¹<https://github.com/wbbsg-uni-mannheim/PyDI>

²<https://github.com/marmleepandya/RAGCleaner>

Attribute	Type	Tasks	LLM difficulty without retrieval
<code>bus_type</code>	text	13	Easy; standard hardware vocabulary
<code>model</code>	text	10	Medium; partially inferable from descriptions
<code>model_number</code>	text	23	Hard; exact SKU, cannot be guessed
<code>read_speed_mb_s</code>	numeric	15	Very hard; rarely in product descriptions
<code>write_speed_mb_s</code>	numeric	10	Very hard; rarely in product descriptions
<code>height_mm</code>	numeric	13	Very hard; must retrieve from KB
<code>width_mm</code>	numeric	12	Very hard; must retrieve from KB
Total		96	

Table 1: Target attributes, evaluation task counts, and expected LLM difficulty without KB access.

This verification is important because it means that any failure is attributable to retrieval or extraction error, not to the absence of the correct value from the knowledge base.

3.2 Dataset and Attribute Selection

Attributes are selected to span a range of difficulty levels and data types, excluding those with fewer than ten recoverable instances (such as `vram_gb` and `chipset_name`) where per-attribute statistics would be unreliable.

The evaluation uses four product offer datasets from WDC Products (Peeters et al. 2024), covering GPUs, SSDs, HDDs, and USB drives, comprising 812 query rows (Dataset 1) and 2,200 KB rows (Datasets 2–4 combined), linked by `cluster_id`.

Seven target attributes are selected to cover a range of difficulty levels, from attributes whose values appear frequently in product descriptions to those that exist only as structured fields in KB records and are essentially impossible to predict without retrieval. Table 1 summarises the selection. The evaluation set is constructed by selecting tasks where the attribute is missing in Dataset 1 and the ground truth is confirmed present in the KB, subject to a minimum of ten tasks per attribute (`MIN_PER_ATTR = 10`) to ensure reliable per-attribute statistics. This yields 96 tasks in total. The evaluation set is sampled once and saved to `eval_set.csv`; every experiment reloads the same fixed set to ensure comparability across configurations.

3.3 System Architecture

The system consists of four components that operate in sequence: an optional retriever (absent in the LLM-only baseline), a prompt builder that formats retrieved candidates into a per-attribute few-shot prompt, a prediction LLM that generates the candidate value, and a response parser that extracts the predicted value from the LLM output or returns UNKNOWN when the model declines to predict. Algorithm 1 shows the full pipeline.

Algorithm 1 RAG-Based Missing Value Imputation Pipeline

Require: Evaluation tasks T (fixed set of row-attribute pairs), knowledge base KB , embedding model M , LLM, re-ranking flag *rerank*

Ensure: Predicted value $\hat{v}$ for each task in T

```
1:  $E_{KB} \leftarrow \text{Encode}_M(KB)$       // both  $KB$  and query embeddings pre-computed, loaded
   from .pt files
2: for all  $(r, a) \in T$  do
3:    $q \leftarrow \text{text}(r)$                 // title, model, brand, product type fields
4:   if  $M$  is LLM-only then
5:      $C \leftarrow \emptyset$                 // no retrieval - Exp 1 baseline
6:   else
7:      $e_q \leftarrow \text{Encode}_M(q)$ 
8:      $C_{20} \leftarrow \text{top-20}(KB, \cos(e_q, E_{KB}))$ 
9:     if rerank then
10:       $C \leftarrow \text{CrossEncoder-rerank}(q, C_{20}, k=5)$ 
11:    else
12:       $C \leftarrow C_{20}[:3]$             // top-3 by cosine similarity - Exp 2
13:    end if
14:  end if
15:   $p \leftarrow \text{BuildPrompt}(r, C, a)$     // per-attribute few-shot prompt
16:   $v \leftarrow \text{LLM}(p)$ 
17:   $\hat{v} \leftarrow \text{ParseResponse}(v)$     // extract VALUE:x or return UNKNOWN
18: end for
```

Retriever: KB embeddings are pre-computed offline and saved as .pt files, with each experiment loading identical tensors to ensure comparability.

Three embedding models are evaluated at increasing representational capacity: **all-MiniLM-L6-v2** (384-dim, 22M parameters), a lightweight model widely used for semantic similarity tasks (Reimers and Gurevych 2019); **BAAI/bge-large-en-v1.5** (1024-dim, 335M parameters), a retrieval-optimised model that ranks among the top performers on the MTEB benchmark (Muennighoff et al. 2023; Xiao et al. 2023); and **OpenAI text-embedding-3-large** (3072-dim)³, an API-based model representing the strongest embedding baseline tested. It should be noted that the use of OpenAI embeddings introduces a partial departure from RetClean’s Scenario 3 privacy constraint: product text is sent to an external API during the embedding phase, though all subsequent retrieval and prediction steps remain local. For a fully privacy-preserving deployment, BGE-large is the recommended configuration. In configurations with re-ranking, a CrossEncoder (**cross-encoder/ms-marco-MiniLM-L6-v2**) reranks the top-20 retrieved candidates to top-5 using pairwise token-level attention. This reranker operates only over the 20 pre-filtered candidates rather than the full KB, making it computationally feasible within the same GPU job as the LLM.

³<https://platform.openai.com/docs/guides/embeddings>

Prompt builder: A separate few-shot prompt is designed for each of the seven target attributes, following the match-then-extract structure described in RetClean (Naeem et al. 2024). The model is first instructed to identify the best matching reference product among the retrieved candidates, then to copy the target attribute value exactly from that product. This two-step instruction reduces the risk of hallucination by grounding extraction in an explicit match decision. For numeric attributes, prompts include explicit warnings against confusing related fields for instance, the `write_speed_mb_s` prompt explicitly cautions against returning the read speed, and the `height_mm` prompt warns against confusing height with width or length.

Where the model cannot identify a sufficiently confident match, it is instructed to return `VALUE:UNKNOWN` rather than produce a speculative answer. This conservative design reflects a deliberate choice. In a data cleaning setting, a confident wrong value propagates silently into downstream systems, whereas an acknowledged abstention can be flagged for human review. This design choice is verified empirically by running a paired experiment for each configuration that removes the `UNKNOWN` option and forces the model to always commit to a prediction. The results of this comparison are reported in Section 4.

Prediction LLM: The primary prediction model across all main experiments is Llama 3.1 8B Instruct (Dubey et al. 2024), served locally via Ollama at `temperature=0` and `seed=42` for deterministic outputs. Llama 3.1 8B was selected because it operates entirely on-premise and falls within the 3B–13B parameter range that RetClean identifies as practical for Scenario 3 deployments. Unlike RetClean’s Scenario 3 setup, the model is not fine-tuned on domain-specific examples; instead, all task adaptation is handled through few-shot prompting. This is a deliberate simplification that isolates the contribution of retrieval quality from any fine-tuning effect.

Two additional experiments replace Llama with GPT-5.4-mini, a cloud-hosted model from OpenAI released in March 2026⁴, to probe whether a stronger LLM closes the remaining performance gap after retrieval is held fixed. These experiments use identical retrieval pipelines (BGE+RR and TE+RR) and identical prompts, isolating the effect of the prediction model. Each LLM experiment is run twice: once with the `UNKNOWN` option available and once with forced prediction, to separately assess extraction capability from abstention behaviour.

PyDI integration: The RAG pipeline is implemented as a `RAGCleaner` class that extends PyDI’s `BaseCleaner` base interface, exposing a `clean(df)` method that accepts a product `DataFrame` and returns it with imputed values. This design makes the component composable with other PyDI stages such as entity matching and schema mapping within the same integration workflow. An additional variant, evaluated in Section 4.4, inserts PyDI’s `DataFusionEngine` between retrieval and LLM prediction. Rather than presenting raw top-5 KB rows to the LLM, the `DataFusionEngine` consolidates them into

⁴Model string: `gpt-5.4-mini`. Accessed via the OpenAI API (<https://platform.openai.com>) in June 2026.

a single record using attribute-specific strategies configured via a `DataFusionStrategy`: majority vote for text attributes, median for numeric fields, and longest-string selection for `model_number`.

3.4 Evaluation Methodology

Standard accuracy: For text attributes, a prediction is correct if the normalised predicted value is a substring of the normalised ground truth or vice versa, capturing common formatting variants. For example, `PCIe 3.0` matches `PCIe 3.0 x16`; `SATA III` matches `SATA 6Gb/s III`; and `WD Blue` matches `WD Blue 6TB Desktop Hard Drive`. For numeric attributes, exact match is adopted: the predicted float must equal the ground truth float exactly.

This stricter numeric criterion is justified empirically: for all numeric evaluation tasks, every non-null value of the target attribute is collected from every KB row sharing the same `cluster_id`. All numeric tasks show zero variation across KB sources, confirming that no tolerance window is needed and exact match is the appropriate metric.

UNKNOWN rate: It reports the fraction of tasks where the model returns `VALUE:UNKNOWN`, reported separately from accuracy. This metric captures over-abstention: a system that abstains on all tasks trivially achieves 0% error on the tasks it attempts, so accuracy and UNKNOWN rate must be read together. A desirable configuration achieves high accuracy with a low UNKNOWN rate, indicating confident and correct predictions. To assess whether UNKNOWN responses reflect genuine uncertainty or model over-caution, each configuration is evaluated both with the UNKNOWN option available and with forced prediction; the two variants are compared in Section 4.

Retrieval quality: Recall@K, Precision@K, NDCG@K, and F1@K are reported at $K \in \{3, 5, 10, 20\}$ for all three embedding models. Recall@K is the primary metric: it measures whether the correct cluster appears anywhere in the top-K candidates, and directly bounds the maximum achievable prediction accuracy. If the correct product is not retrieved, no prompt engineering or LLM capability can recover the right value.

4 Experimental Results

4.1 Overall Results

Table 2 presents overall standard accuracy and abstention rate across all seven configurations under two evaluation regimes: abstention permitted (UNKNOWN allowed) and forced prediction (UNKNOWN disallowed).

Retrieval substantially and consistently outperforms LLM-only prediction under permitted abstention. The LLM-only baseline reaches only 15.6% accuracy, with nearly half of all tasks returning UNKNOWN, a direct consequence of attributes such as model numbers and physical dimensions carrying no signal in the product description for the model to draw on. Introducing MiniLM-based retrieval more than doubles accuracy

Exp	Configuration	Permitted		Forced	
		Acc	Abst	Acc	Abst
1	LLM-only	15.6%	44.8%	13.5%	35.4%
2	RAG + MiniLM	36.5%	17.7%	37.5%	6.2%
3	MiniLM + RR	68.8%	8.3%	66.7%	6.2%
4	BGE + RR	72.9%	3.1%	69.8%	2.1%
5	TE + RR	71.9%	2.1%	70.8%	2.1%
6	BGE + RR (GPT-5.4-mini)	60.4%	34.4%	76.0%	1.0%
7	TE + RR (GPT-5.4-mini)	63.5%	27.1%	77.1%	1.0%

Table 2: Standard accuracy (Acc) and abstention rate (Abst) under permitted abstention and forced prediction. Exps 1–5: Llama 3.1 8B Instruct. Exps 6–7: GPT-5.4-mini. RR = CrossEncoder reranker.

to 36.5%, and adding CrossEncoder re-ranking on top pushes it further to 68.8%, a far larger jump than retrieval alone produced. This asymmetry implies that candidate *precision*, not embedding *quality*, is the primary bottleneck once retrieval is already in place: cosine similarity over pooled embeddings is sufficient to narrow the search space, but distinguishing the correct product from near-identical variants requires the pairwise comparison a CrossEncoder provides. Upgrading from MiniLM to BGE-large adds 4.1 points to 72.9%; switching further to OpenAI TE-3-large yields 71.9%, a marginal regression suggesting that retrieval quality beyond BGE no longer constrains accuracy and the bottleneck has shifted to extraction.

A more unexpected pattern emerges when the prediction LLM is varied. On the surface, GPT-5.4-mini looks like a clear regression: under permitted abstention it trails every Llama RAG configuration by 8-12 points, abstaining on 27–34% of tasks compared to Llama’s 2-3%. Table 2 (Forced columns) reveals that this is not a capability gap at all. Once abstention is disallowed, GPT-5.4-mini’s accuracy jumps by +15.6 and +13.5 percentage points, producing the two highest results across all seven configurations. The conclusion is that GPT-5.4-mini already extracts the correct value in most cases where it previously abstained; the uncertainty instruction, calibrated around Llama’s more decisive response style, is simply miscalibrated for GPT-5.4-mini, which treats it as licence to hedge far more readily than intended.

4.2 Per-Attribute Results

Table 3 shows per-attribute standard accuracy for the five Llama configurations under permitted abstention.

Numeric attributes are not strictly impossible for LLM-only. An important correction to the naive expectation: numeric attributes do not score exactly 0% for LLM-only prediction. `read_speed_mb_s` reaches 13.3% and `write_speed_mb_s` reaches 30.0%. Man-

Attribute	Type	Exp 1	Exp 2	Exp 3	Exp 4	Exp 5
bus_type	text	53.8	84.6	76.9	92.3	84.6
model	text	10.0	40.0	50.0	50.0	40.0
model_number	text	4.3	21.7	65.2	69.6	69.6
read_speed_mb_s	numeric	13.3	40.0	73.3	80.0	80.0
write_speed_mb_s	numeric	30.0	40.0	90.0	90.0	80.0
height_mm	numeric	0.0	15.4	69.2	69.2	76.9
width_mm	numeric	8.3	25.0	58.3	58.3	66.7

Table 3: Per-attribute standard accuracy (%) for Llama 3.1 8B Instruct across five retrieval configurations (permitted abstention).

Exp 1: LLM-only; Exp 2: RAG-MiniLM; Exp 3: MiniLM+Reranker; Exp 4: BGE+Reranker; Exp 5: TE+Reranker.

ual inspection reveals that these are coincidental matches: the LLM produces common SSD speeds such as 550 MB/s or 520 MB/s that happen to match the ground truth for several SATA SSD products in the evaluation set, since these are industry-standard values for that product class. LLM-only numeric accuracy is therefore inflated by distributional coincidence rather than product-specific knowledge. `height_mm` scores exactly 0.0%, confirming that physical dimensions carry no distributional regularity the LLM could exploit.

CrossEncoder re-ranking resolves model_number. `model_number` shows the most dramatic improvement from re-ranking: from 21.7% (Exp 2) to 65.2% (Exp 3), a 43.5-point gain. This is consistent with the motivating design of RetClean’s two-stage pipeline (Naeem et al. 2024): exact SKUs are indistinguishable by cosine similarity over pooled embeddings when near-identical variants exist, but the CrossEncoder’s pairwise token-level attention separates them more reliably.

4.3 LLM Comparison: Llama vs GPT-5.4-mini

Table 4 compares Llama 3.1 8B Instruct and GPT-5.4-mini on BGE+RR under forced prediction, isolating extraction capability from abstention behaviour.

GPT-5.4-mini outperforms Llama overall (76.0% vs 69.8%), with the advantage concentrated almost entirely in numeric attributes (84.0% vs 69.4%). For text attributes the two models are nearly equivalent (67.2% vs 68.1%), with the difference driven primarily by `bus_type` where GPT-5.4-mini scores only 61.5% versus Llama’s 84.6%. This is the only attribute where Llama is meaningfully stronger, and it is also the attribute with the most standard vocabulary, suggesting GPT-5.4-mini’s conservatism shows up even in committed predictions on ambiguous bus type variants. Both models achieve identical performance on `model_number` (69.6%), confirming that this attribute’s ceiling is set by retrieval precision rather than LLM capability: when the CrossEncoder surfaces the wrong near-identical SKU, neither model can recover the correct value.

Attribute	Type	Llama 3.1 8B	GPT-5.4-mini
bus_type	text	84.6%	61.5%
model	text	50.0%	70.0%
model_number	text	69.6%	69.6%
read_speed_mb_s	numeric	80.0%	86.7%
write_speed_mb_s	numeric	70.0%	100.0%
height_mm	numeric	69.2%	76.9%
width_mm	numeric	58.3%	75.0%
Overall		69.8%	76.0%
Numeric avg.		69.4%	84.0%
Text avg.		68.1%	67.2%

Table 4: Per-attribute standard accuracy under forced prediction on BGE+RR

The numeric advantage tells the more interesting story. Storage speeds and physical dimensions require faithful copying of an exact value from the retrieved KB row. For Llama, field confusion: copying the read speed when asked for the write speed, or confusing height with width, is the primary failure mode on numeric attributes despite explicit prompt warnings. GPT-5.4-mini reduces this error class significantly, achieving 100.0% on `write_speed_mb_s` versus Llama’s 70.0%, and gains of 6–17 points across all four numeric attributes. The evidence suggests that instruction-following fidelity, not retrieval quality, is the remaining bottleneck for numeric extraction once the correct product is retrieved.

4.4 PyDI Data Fusion Variant

Table 5 compares the standard TE+RR pipeline (Exp 5, Llama) with a variant that inserts PyDI’s `DataFusionEngine` before LLM prediction. Both configurations use identical retrieval; the only difference is that the fusion variant consolidates the top-5 KB candidates into a single clean record before the LLM sees them.

Fusion reduces the UNKNOWN rate from 2.1% to 1.0% and leaves most numeric attributes unchanged, confirming that median aggregation over the top-5 is a reliable strategy for numeric fields when those candidates contain the correct product. The cost appears for text attributes: `model_number` drops by 26.1 percentage points and `bus_type` by 15.4 points. For `model_number`, the longest-string strategy frequently selects a verbose variant string whose format does not match the shorter ground-truth SKU. For `bus_type`, majority vote fails when the top-5 candidates represent products with different interface types, replacing the correct minority value with a more common but wrong one. The pattern reveals a fundamental limitation of schema-level aggregation strategies: they cannot distinguish format variation, different ways of writing the same value, from semantic variation, and genuinely different values from different products.

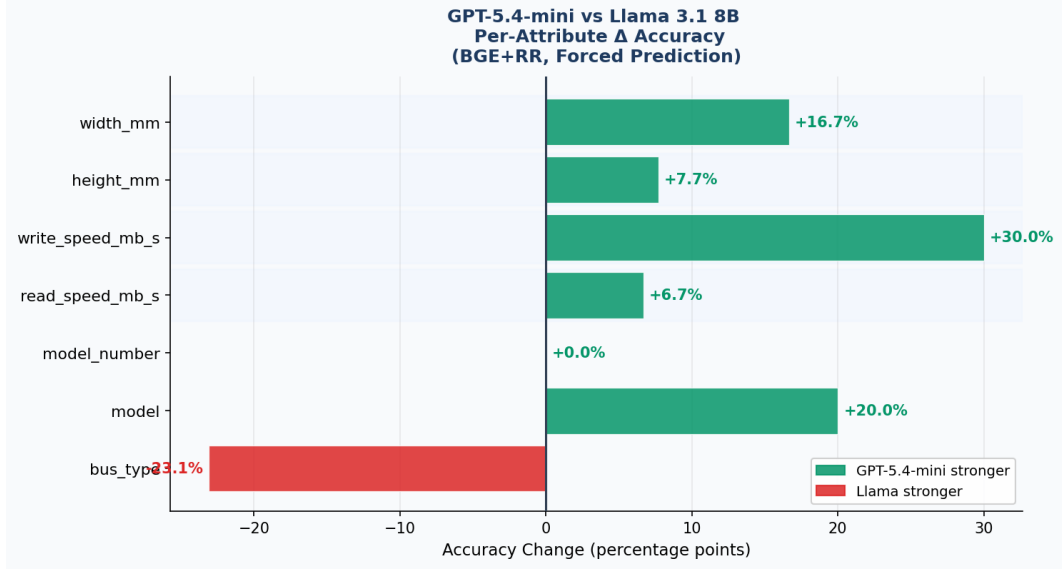


Figure 1: Per-attribute accuracy difference between GPT-5.4-mini and Llama 3.1 8B under forced prediction on BGE+RR.

4.5 Retrieval Quality

Table 6 reports retrieval metrics at $K=5$ and $K=20$ for all three embedding models, computed from saved embedding tensors without re-running the LLM.

MiniLM’s Recall@5 of 0.708 directly explains the Exp 2–3 accuracy gap: for 29.2% of tasks the correct cluster never appears in the top-5, making recovery impossible regardless of LLM capability. BGE and OpenAI both achieve Recall@5 of 0.833, but OpenAI’s higher Precision@5 (0.327 vs 0.287) and NDCG@5 (0.710 vs 0.632) mean it ranks the correct cluster higher, a meaningful advantage for the CrossEncoder, which operates on the ranked top-5. OpenAI achieves perfect Recall@20 while BGE misses one task in a hundred; this marginal gap is often compensated by the CrossEncoder, explaining why TE does not consistently outperform BGE in end-to-end accuracy.

5 Error Analysis

Of the 96 evaluation tasks in Exp 5 (TE+RR + Llama), 69 are correct and 27 are wrong. Only 2 of the 27 failures are abstentions (UNKNOWN); the remaining 25 are confident wrong predictions.

Ambiguous or inconsistent ground truth (30%): The largest failure class reflects annotation quality rather than pipeline failure. Several ground truth values are short abbreviations or internal codes that do not match how any retrieved product describes itself: for example, GT `model` = ISS is an enterprise abbreviation, and GT `model` = 220S is an internal product code. In other cases the ground truth uses a different field

Attribute	Type	TE+RR (Exp 5)	PyDI Fusion
bus_type	text	84.6%	69.2%
model	text	40.0%	40.0%
model_number	text	69.6%	43.5%
read_speed_mb_s	numeric	80.0%	73.3%
write_speed_mb_s	numeric	80.0%	80.0%
height_mm	numeric	76.9%	76.9%
width_mm	numeric	66.7%	66.7%
Overall		71.9%	62.5%
UNKNOWN rate		2.1%	1.0%

Table 5: Standard accuracy and UNKNOWN rate: TE+RR baseline vs PyDI DataFusionEngine variant. Both use Llama 3.1 8B Instruct with permitted abstention.

Embedding Model	R@5	P@5	NDCG@5	R@20	NDCG@20
MiniLM (384-dim)	0.708	0.196	0.480	0.938	0.537
BGE (1024-dim)	0.833	0.287	0.632	0.990	0.646
OpenAI (3072-dim)	0.833	0.327	0.710	1.000	0.728

Table 6: Retrieval quality at $K=5$ (CrossEncoder input) and $K=20$ (initial pool). R = Recall, P = Precision. OpenAI achieves perfect Recall@20 across all 96 tasks.

than the LLM extracts: GT `model = Barracuda 7200.12` is filled as a product family name while the retrieved KB row lists it as a model number.

Field confusion (22%): The correct product is retrieved but the LLM copies the wrong field. The clearest examples involve numeric attributes in close proximity: for one GPU product, the write speed is predicted as 2.5 (the card thickness in mm), and height and width are swapped between two different dimension fields. Despite explicit prompt warnings against this, Llama 3.1 8B confuses co-occurring numeric attributes when multiple values of similar magnitude appear in the candidate text. A constrained output format that presents only the target attribute’s value to the LLM would likely eliminate this class.

Numeric near-miss (22%): The retriever returns a product that is very similar but not identical to the correct one, and the LLM faithfully copies the wrong value. Examples include read speed 540 MB/s predicted as 530 MB/s, and width 119.3 mm predicted as 119 mm. These are not extraction errors the LLM correctly reads the retrieved value but retrieval errors where the CrossEncoder promotes a near-duplicate product over the correct one. This failure class would benefit from a fine-tuned product matching model that can distinguish near-identical hardware variants.

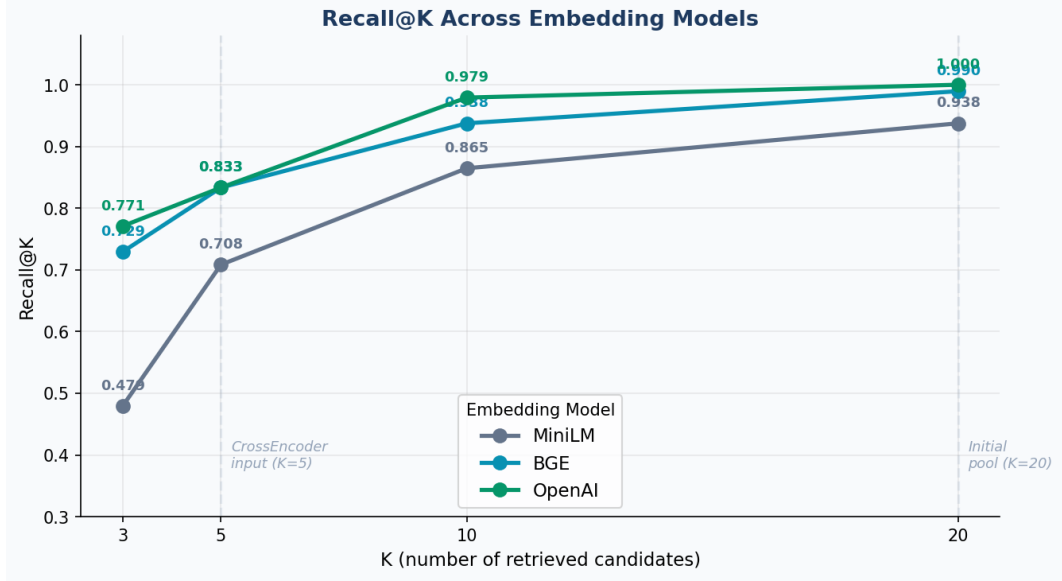


Figure 2: Recall@K for all three embedding models as K increases from 3 to 20

Near-identical SKU confusion (19%): Retrieval returns a product from the same family with a different model number suffix. For example, GV-N3080GAMING OC-10GD (GAMING variant) is confused with GV-N3080EAGLE-OC-10GD (EAGLE variant), and NE6166SS18J9-161F is predicted as NE51660018J9-165F with a single character transposition. Cosine similarity and CrossEncoder re-ranking cannot reliably separate these variants because their textual descriptions are nearly identical. The two genuine abstentions in this experiment (UNKNOWN) both occur in this category, reflecting cases where the model correctly recognises the ambiguity rather than committing to the wrong SKU.

Version mismatch (7%): Two bus type errors involve interface version confusion (USB 3.0 predicted as USB 3.1, PCIe 3.0 x16 as PCIe 3.0 x8), reflecting retrieval of a slightly different product generation rather than an extraction failure.

6 Conclusions and Future Work

Across 96 evaluation tasks and seven attribute types, the experiments show that retrieval is not uniformly beneficial: its value depends on attribute type, retrieval precision, and prediction LLM calibration. Four concrete conclusions follow.

Retrieval with re-ranking is essential, not optional. LLM-only prediction achieves 15.6% accuracy. Adding retrieval alone reaches 36.5%; adding CrossEncoder re-ranking on top reaches 68.8%. The 32-point gain from re-ranking dwarfs the gain from retrieval alone, confirming that candidate precision not KB coverage is the binding constraint. For numeric attributes, retrieval is the difference between near-zero and 70–100%: these values exist verbatim in KB structured fields but carry no signal in product

descriptions.

The bottleneck shifts as the pipeline improves. With MiniLM retrieval, 29.2% of tasks never see the correct product in the top-5, making retrieval recall the bottleneck. With BGE+RR, Recall@20 reaches 99% but 22% of remaining failures are field confusion and 22% are numeric near-misses, shifting the bottleneck to extraction quality. With GPT-5.4-mini under forced prediction, numeric extraction improves to 84.0% versus Llama’s 69.4%.

Prompt-model calibration matters as much as model capability. GPT-5.4-mini under permitted abstention scores only 60–63%, trailing every Llama RAG configuration. Under forced prediction it scores 76–77%, the best results overall. This reversal reveals a calibration mismatch: the uncertainty instruction designed for Llama causes GPT-5.4-mini to abstain on tasks it could answer correctly. The finding also exposes a fundamental tension within the Scenario 3 privacy constraint: the strongest results require GPT-5.4-mini as the prediction model, which sends product data to an external OpenAI API, directly undermining the privacy motivation. OpenAI embeddings (text-embedding-3-large) introduce a further privacy departure during the retrieval phase. For a deployment where no data may leave the organisation, BGE+RR + Llama at 72.9% is the appropriate configuration, accepting a 4-point accuracy penalty in exchange for full on-premise operation.

Thirty percent of failures reflect dataset quality, not the pipeline. The largest single error class (30%) is ambiguous or inconsistent ground truth: abbreviations such as ISS, internal codes such as 220S, and cross-field annotation inconsistencies that no retrieval or extraction improvement can resolve.

Returning to the three questions from the introduction: retrieval improves substantially over LLM-only prediction, with the gain concentrated in attributes that carry no signal in product descriptions; CrossEncoder re-ranking delivers the single largest improvement in the two-stage pipeline; and GPT-5.4-mini closes the remaining gap under forced prediction, but only at the cost of external API calls that directly conflict with the Scenario 3 privacy constraint.

7 Future work

A domain-specific product matching model fine-tuned on near-duplicate hardware pairs would address the 41% of failures from SKU confusion and numeric near-misses. LLM-specific prompt calibration whether for GPT-5.4-mini or a capable open-weights model such as Llama 3.1 70B would resolve the abstention mismatch while preserving Scenario 3 privacy. Extending the PyDI fusion strategy with attribute-aware conflict resolution, for instance distinguishing format variation from semantic variation before aggregating, could recover the accuracy lost on `model_number` and `bus_type` without sacrificing the UNKNOWN reduction.

References

- Bodensohn, J.-M., U. Brackmann, L. Vogel, M. Urban, A. Sanghi, and C. Binnig (2024). LLMs for data engineering on enterprise data. In *Proceedings of the VLDB 2024 Workshop on Tabular Data Analysis (TaDA)*.
- Douze, M. et al. (2024). The FAISS library. *arXiv preprint arXiv:2401.08281*.
- Dubey, A. et al. (2024). The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*.
- Gao, Y., Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, and H. Wang (2023). Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*.
- Izacard, G. and E. Grave (2021). Leveraging passage retrieval with generative models for open domain question answering. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics (EACL)*, pp. 874–880.
- Karpukhin, V., B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W. tau Yih (2020). Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 6769–6781.
- Khattab, O. and M. Zaharia (2020). ColBERT: Efficient and effective passage search via contextualized late interaction over BERT. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pp. 39–48.
- Lewis, P., E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. tau Yih, T. Rocktäschel, S. Riedel, and D. Kiela (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, Volume 33, pp. 9459–9474.
- Li, P., Y. He, D. Yashar, W. Cui, S. Ge, J. Macke, H. Zhang, D. Zhang, and S. Chaudhuri (2023). Table-GPT: Table-tuned GPT for diverse table tasks. *arXiv preprint arXiv:2310.09263*.
- Muennighoff, N., N. Tazi, L. Magne, and N. Reimers (2023). MTEB: Massive text embedding benchmark. In *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics (EACL)*, pp. 2014–2037.
- Naeem, Z. A., M. S. Ahmad, M. Eltabakh, M. Ouzzani, and N. Tang (2024, August). Retclean: Retrieval-based data cleaning using LLMs and data lakes. *Proceedings of the VLDB Endowment* 17(12), 4421–4424.
- Narayan, A., I. Chami, L. Orr, S. Arora, and C. Ré (2022). Can foundation models wrangle your data? *Proceedings of the VLDB Endowment* 16(4), 738–746.

- Nogueira, R. and K. Cho (2019). Passage re-ranking with BERT. *arXiv preprint arXiv:1901.04085*.
- Papadakis, G., E. Ioannou, E. Thanos, and T. Palpanas (2021). The four generations of entity resolution. In *Proceedings of the 37th IEEE International Conference on Data Engineering (ICDE)*, pp. 2485–2497.
- Peeters, R. and C. Bizer (2023). Using ChatGPT for entity matching. In *New Trends in Database and Information Systems*, Volume 1850 of *Communications in Computer and Information Science*, pp. 221–230. Springer.
- Peeters, R., R. C. Der, and C. Bizer (2024). WDC products: A multi-dimensional entity matching benchmark. In *Proceedings of the 27th International Conference on Extending Database Technology (EDBT)*.
- Rahm, E. and H. H. Do (2000). Data cleaning: Problems and current approaches. *IEEE Data Engineering Bulletin* 23(4), 3–13.
- Reimers, N. and I. Gurevych (2019). Sentence-BERT: Sentence embeddings using siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 3982–3992.
- Robertson, S. and H. Zaragoza (2009). The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval* 3(4), 333–389.
- Thakur, N., N. Reimers, A. Rücklé, A. Srivastava, and I. Gurevych (2021). BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*.
- van Buuren, S. (2018). *Flexible Imputation of Missing Data* (2nd ed.). Boca Raton: Chapman & Hall/CRC.
- Xiao, S., Z. Liu, P. Zhang, N. Muennighoff, D. Lian, and J.-Y. Nie (2023). C-Pack: Packaged resources to advance general chinese embedding. *arXiv preprint arXiv:2309.07597*.
- Zhang, A., C. Chai, G. Li, and N. Tang (2023). Large language models as data preprocessors. *arXiv preprint arXiv:2308.16361*.
- Zhao, P., H. Zhang, Q. Yu, Z. Wang, Y. Geng, F. Fu, L. Yang, W. Zhang, and B. Cui (2024). Retrieval-augmented generation for AI-generated content: A survey. *arXiv preprint arXiv:2402.19473*.

Ehrenwörtliche Erklärung

Ich versichere, dass ich die beiliegende Bachelor-, Master-, Seminar-, oder Projektarbeit ohne Hilfe Dritter und ohne Benutzung anderer als der angegebenen Quellen und in der untenstehenden Tabelle angegebenen Hilfsmittel angefertigt und die den benutzten Quellen wörtlich oder inhaltlich entnommenen Stellen als solche kenntlich gemacht habe. Diese Arbeit hat in gleicher oder ähnlicher Form noch keiner Prüfungsbehörde vorgelegen. Ich bin mir bewusst, dass eine falsche Erklärung rechtliche Folgen haben wird.

Declaration of Used AI Tools

Tool	Purpose	Where?	Useful?
Claude Sonnet 4.6	Report	Throughout	++
ChatGPT	Report	Throughout	+
Perplexity	Research	Throughout	+
Claude Sonnet 4.6	Code Generation and Debugging	Throughout	++



Mannheim, den July 1, 2026